

RISC-V Processor Verification Report

Objective

- ☐ Verification of RISC-V pipeline modules.
- ☐ Functional verification using SystemVerilog testbenches.
- ☐ Assertion-Based Verification (SVA).
- ☐ Functional Coverage Analysis.
- ☐ Simulations executed using Aldec Riviera-PRO 2025.04 in EDA Playground.

Table of Contents

Verification Methodology	3
Verification Summary Table	4
1. Program Counter (PC) Verification	5
2. PC Offset Verification	8
3. Arithmetic Logic Unit (ALU) Verification	10
4 ALU Control Verification	14
5. Adder Verification	17
6. Branch Decision Verification	19
7. Control Unit Verification	22
8 Control Mux Verification	24
9. Data Memory Verification	26
10. EX/MEM Pipeline Register Verification	29
11. Forwarding Unit Verification	32
12. Hazard Detection Unit Verification	34
13. ID/EX Pipeline Register Verification	36
14. IF/ID Pipeline Register Verification	39
15. Immediate Generator (IMM_GEN) Verification	41
16. Instruction Memory Verification	43
17. MEM/WB Pipeline Register Verification	45
18. MUX_WB Verification	48
19. MUX Verification	50
20. MUXSRC1 Verification	52
21. MUXSRC2 Verification	54
22. MUXSRCIMM Verification	56
23. REG_FILE Verification	58
24. RISC_TOP Verification	62
Future Work	65

Verification Methodology

The verification phase of the RISC-V processor project was carried out using a structured unit-level verification methodology. Each RTL module was verified independently before proceeding to top-level integration.

Initially, dedicated SystemVerilog testbenches were developed for every module. Multiple directed test cases were created to validate normal operation, boundary conditions, reset behavior, hazard conditions, pipeline control scenarios, and exceptional cases. Functional correctness was confirmed through simulation results and waveform analysis.

After successful testcase execution, functional coverage models were implemented using SystemVerilog covergroups, coverpoints, and cross coverage. Simulations were executed on Aldec Riviera-PRO 2025 through EDA Playground. Coverage reports were analyzed and additional testcases were introduced whenever required to ensure complete functional coverage. All verified modules achieved 100% functional coverage.

Following functional coverage closure, Assertion-Based Verification (ABV) was incorporated. Sequential modules such as PC, IF_ID, ID_EX, EX_MEM, MEM_WB, Register File, and other pipeline registers were verified using SystemVerilog Assertions (SVA). Combinational modules such as ALU Control, Control Unit, Hazard Detection Unit, Forwarding Unit, and Multiplexers were verified using immediate assertions embedded within testcase execution. This approach was adopted due to simulator limitations associated with assertion support in the EDA Playground environment.

All assertions were executed successfully without failures after testcase refinement and debugging. This provided an additional layer of confidence beyond functional coverage by automatically checking critical design properties during simulation.

At the current stage, unit verification of all RTL modules has been completed successfully. Functional coverage closure has been achieved, and assertion-based verification has been completed for all verified modules.

The next phase of work focuses on:

1. Collection and analysis of detailed code coverage metrics.
2. Statement, branch, toggle, condition, and expression coverage closure.
3. Top-level processor verification refinement.
4. Constrained-random verification and advanced verification methodologies.
5. Final verification signoff and documentation.

Verification Summary Table

Module	Functional Coverage	Assertions	Status
Adder	100%	PASS	PASS
ALU	100%	PASS	PASS
ALU Control	100%	PASS	PASS
Branch Decision	100%	PASS	PASS
Control	100%	PASS	PASS
Control Mux	100%	PASS	PASS
Data Memory	100%	PASS	PASS
EX_MEM	100%	PASS	PASS
Forwarding Unit	100%	PASS	PASS
Hazard Detection Unit	100%	PASS	PASS
ID_EX	100%	PASS	PASS
IF_ID	100%	PASS	PASS
Immediate Generator	100%	PASS	PASS
Instruction Memory	100%	PASS	PASS
MEM_WB	100%	PASS	PASS
MUX	100%	PASS	PASS
MUX_WB	100%	PASS	PASS
MUX_SRC1	100%	PASS	PASS
MUX_SRC2	100%	PASS	PASS
MUX_SRCIMM	100%	PASS	PASS
Program Counter (PC)	100%	PASS	PASS
PC Offset	100%	PASS	PASS
Register File	100%	PASS	PASS
RISC-V Top Module	100%	PASS	PASS

1. Program Counter (PC) Verification

Module Description

The Program Counter (PC) module stores the current instruction address and updates it on every clock cycle. It supports reset functionality and pipeline stalling through the pc_write signal.

Features Verified

- Synchronous reset operation
- Normal PC update operation
- Stall condition handling
- Multiple sequential PC updates
- Large address handling

Testcases Executed

TC No	Description	Verification Objective	Result
TC1	Reset Verification	Verify PC resets to zero when reset is asserted	PASS
TC2	Normal PC Update Verification	Verify PC updates with next_pc when pc_write is enabled	PASS
TC3	Stall Verification	Verify PC retains previous value when pc_write is disabled	PASS
TC4	Second PC Update Verification	Verify multiple updates occur correctly across cycles	PASS
TC5	Large Address Verification	Verify PC correctly handles large 32-bit address values	PASS
TC6	Reset Recovery Verification	Verify PC returns to zero after reset is reasserted	PASS

Assertions Used

Assertion	Purpose	Result
Reset Assertion	Verify PC resets to zero on reset	PASS
PC Update Assertion	Verify PC updates when pc_write=1	PASS
Stall Assertion	Verify PC holds previous value when pc_write=0	PASS

Functional Coverage

Coverpoint Bins Covered

rst reset_on, reset_off

pc_write write_enable, write_disable

Functional Coverage Achieved: 100%

Verification Result

```
39  # KERNEL: TC1 : RESET
40  # KERNEL: SVA PASS : UPDATE
41  # KERNEL: SVA PASS : STALL
42  # KERNEL: PASS : TC1
43  # KERNEL: ASSERT PASS : RESET
44  # KERNEL:
45  # KERNEL: TC2 : NORMAL UPDATE
46  # KERNEL: SVA PASS : RESET
47  # KERNEL: SVA PASS : RESET
48  # KERNEL: SVA PASS : STALL
49  # KERNEL: PASS : TC2
50  # KERNEL: ASSERT PASS : UPDATE
51  # KERNEL:
52  # KERNEL: TC3 : STALL
53  # KERNEL: SVA PASS : RESET
54  # KERNEL: SVA PASS : UPDATE
55  # KERNEL: SVA PASS : UPDATE
56  # KERNEL: PASS : TC3
57  # KERNEL: ASSERT PASS : STALL
58  # KERNEL:
59  # KERNEL: TC4 : SECOND UPDATE
60  # KERNEL: SVA PASS : RESET
61  # KERNEL: SVA PASS : STALL
62  # KERNEL: SVA PASS : STALL
63  # KERNEL: PASS : TC4
64  # KERNEL: ASSERT PASS : SECOND UPDATE
65  # KERNEL:
66  # KERNEL: TC5 : LARGE VALUE
67  # KERNEL: SVA PASS : RESET
68  # KERNEL: SVA PASS : UPDATE
69  # KERNEL: SVA PASS : STALL
70  # KERNEL: PASS : TC5
71  # KERNEL: ASSERT PASS : LARGE VALUE
72  # KERNEL:
73  # KERNEL: TC6 : RESET AGAIN
74  # KERNEL: SVA PASS : UPDATE
75  # KERNEL: SVA PASS : UPDATE
76  # KERNEL: SVA PASS : STALL
77  # KERNEL: PASS : TC6
78  # KERNEL: ASSERT PASS : RESET AGAIN
```

- Functional Coverage : 100%

```

205 # MODULE - work.tb_pc
206 #
207 #
208 # SUMMARY
209 # =====
210 # | Coverage Type | Weight | Hits/Total |
211 # =====
212 # | Covergroup Coverage | 1 | 100.000% |
213 # |-----|-----|-----|
214 # | Types | 1 / 1 |
215 # =====
216 # WEIGHTED AVERAGE: 100.000%
217 #
218 #
219 # COVERGROUP COVERAGE
220 # =====
221 # | Covergroup | Hits | Goal / | Status |
222 # | | | At Least | |
223 # =====
224 # | TYPE /tb_pc/pc_cg | 100.000% | 100.000% | Covered |
225 # =====
226 # | INSTANCE PC_COVERAGE | 100.000% | 100.000% | Covered |
227 # |-----|-----|-----|
228 # | COVERPOINT PC_COVERAGE::cp_rst | 100.000% | 100.000% | Covered |
229 # |-----|-----|-----|
230 # | bin reset_on | 3 | 1 | Covered |
231 # | bin reset_off | 4 | 1 | Covered |
232 # |-----|-----|-----|
233 # | COVERPOINT PC_COVERAGE::cp_write | 100.000% | 100.000% | Covered |
234 # |-----|-----|-----|
235 # | bin write_enable | 5 | 1 | Covered |
236 # | bin write_disable | 2 | 1 | Covered |
237 # =====
238 #
239 exit
240 # KERNEL:
241 # KERNEL: Final Coverage = 100.00%
242 # VSIM: Simulation has finished.

```

- Module Status : VERIFIED ✓

2. PC Offset Verification

Module Description

The PC Offset module calculates the target address by adding the current Program Counter (PC) value with the immediate offset generated from the instruction. It is primarily used for branch and jump target address generation.

Testcases Executed

TC No	Description	Result
TC1	Zero PC and Zero Immediate Verification	PASS
TC2	Positive Offset Verification	PASS
TC3	Negative Offset Verification	PASS
TC4	Large Value Addition Verification	PASS
TC5	Overflow Verification	PASS
TC6	Dynamic Input Switching Verification	PASS

Assertions Used

Assertion	Purpose	Result
Zero Addition Assertion	Verify $0 + 0$ operation	PASS
Positive Offset Assertion	Verify positive immediate addition	PASS
Negative Offset Assertion	Verify negative immediate addition	PASS
Large Value Assertion	Verify large operand addition	PASS
Overflow Assertion	Verify wrap-around behavior during overflow	PASS
Dynamic Switching Assertion	Verify output update correctly for changing input	PASS

Functional Coverage

Coverpoint Bins Covered

pc	zero_pc, small_pc, large_pc
imm	zero_imm, positive_imm, negative_imm, large_imm

```

# KERNEL:
# KERNEL: TC1 : ZERO + ZERO
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : ZERO
# KERNEL:
# KERNEL: TC2 : POSITIVE OFFSET
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : POSITIVE OFFSET
# KERNEL:
# KERNEL: TC3 : NEGATIVE OFFSET
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : NEGATIVE OFFSET
# KERNEL:
# KERNEL: TC4 : LARGE VALUES
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : LARGE VALUES
# KERNEL:
# KERNEL: TC5 : OVERFLOW
# KERNEL: PASS : TC5
# KERNEL: ASSERT PASS : OVERFLOW
# KERNEL:
# KERNEL: TC6 : DYNAMIC SWITCHING
# KERNEL: ASSERT PASS : DYNAMIC-1
# KERNEL: ASSERT PASS : DYNAMIC-2
# KERNEL: PASS : TC6
# KERNEL:
# KERNEL: PC_OFFSET VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%

```

```

190 # MODULE - work.tb_pc_offset
191 #
192 #
193 # SUMMARY
194 #
195 # | Coverage Type | Weight | Hits/Total |
196 # |-----|-----|-----|
197 # | Covergroup Coverage | 1 | 100.000% |
198 # |-----|-----|-----|
199 # | Types | 1 / 1 |
200 #
201 # WEIGHTED AVERAGE: 100.000%
202 #
203 #
204 # COVERGROUP COVERAGE
205 #
206 # | Covergroup | Hits | Goal / | Status |
207 # | | | At Least | |
208 # |-----|-----|-----|
209 # | TYPE /tb_pc_offset/pc_offset_cg | 100.000% | 100.000% | Covered |
210 # |-----|-----|-----|
211 # | INSTANCE PC_OFFSET_COVERAGE | 100.000% | 100.000% | Covered |
212 # |-----|-----|-----|
213 # | COVERPOINT PC_OFFSET_COVERAGE::cp_pc | 100.000% | 100.000% | Covered |
214 # |-----|-----|-----|
215 # | bin zero_pc | 1 | 1 | Covered |
216 # | bin small_pc | 2 | 1 | Covered |
217 # | bin large_pc | 4 | 1 | Covered |
218 # |-----|-----|-----|
219 # | COVERPOINT PC_OFFSET_COVERAGE::cp_imm | 100.000% | 100.000% | Covered |
220 # |-----|-----|-----|
221 # | bin zero_imm | 1 | 1 | Covered |
222 # | bin positive_imm | 3 | 1 | Covered |
223 # | bin large_imm | 3 | 1 | Covered |
224 # |-----|-----|-----|
225 #
226 exit
227 # KERNEL:
228 # KERNEL: Final Coverage = 100.00%
229 # KERNEL: Coverage = 100.00%

```

3. Arithmetic Logic Unit (ALU) Verification

Module Description

The Arithmetic Logic Unit (ALU) performs arithmetic, logical, shift, comparison, branch evaluation, load/store address calculation, and special RISC-V operations. It also generates the branch condition signal used by the branch decision logic.

Testcases Executed

TC No	Description	Result
TC1	ADD Operation Verification	PASS
TC2	SUB Operation Verification	PASS
TC3	AND Operation Verification	PASS
TC4	OR Operation Verification	PASS
TC5	XOR Operation Verification	PASS
TC6	SLL Operation Verification	PASS
TC7	SRL Operation Verification	PASS
TC8	SRA Operation Verification	PASS
TC9	SLT Operation Verification	PASS
TC10	SLTU Operation Verification	PASS
TC11	Load/Store Address Calculation Verification	PASS
TC12	BEQ Branch Verification	PASS
TC13	BNE Branch Verification	PASS
TC14	BLT Branch Verification	PASS
TC15	BGE Branch Verification	PASS
TC16	LUI Operation Verification	PASS
TC17	JAL Operation Verification	PASS
TC18	Default Case Verification	PASS
TC19	ADDI Operation Verification	PASS
TC20	BLTU Branch Verification	PASS
TC21	BGEU Branch Verification	PASS

Assertions Used

Assertion	Purpose	Result
ADD Assertion	Verify addition operation	PASS
SUB Assertion	Verify subtraction operation	PASS
AND Assertion	Verify bitwise AND operation	PASS
OR Assertion	Verify bitwise OR operation	PASS
XOR Assertion	Verify bitwise XOR operation	PASS
Shift Assertions	Verify SLL, SRL and SRA operations	PASS
Comparison Assertions	Verify SLT and SLTU operations	PASS
Load/Store Assertion	Verify address calculation logic	PASS
Branch Assertions	Verify BEQ, BNE, BLT, BGE, BLTU and BGEU conditions	PASS
LUI Assertion	Verify immediate load operation	PASS
JAL Assertion	Verify JAL operation output	PASS
Default Assertion	Verify default case output	PASS

Functional Coverage

Coverpoint Bins Covered

operation ADD, SUB, SLL, SRL, SRA, SLT, SLTU, XOR, OR, AND

operation ADDI, Load/Store Operations

operation BEQ, BNE, BLT, BGE, BLTU, BGEU

operation LUI, JAL

operation Default Case

Functional Coverage Achieved: 100%

Verification Result

- Directed Testcases : PASS
- Assertions : PASS
- Functional Coverage : 100%
- Module Status : VERIFIED ✓

```
# KERNEL: TC1 : ADD
# KERNEL: PASS : ADD
# KERNEL: ASSERT PASS : ADD
# KERNEL:
# KERNEL: TC2 : SUB
# KERNEL: PASS : SUB
# KERNEL: ASSERT PASS : SUB
# KERNEL:
# KERNEL: TC3 : AND
# KERNEL: PASS : AND
# KERNEL: ASSERT PASS : AND
# KERNEL:
# KERNEL: TC4 : OR
# KERNEL: PASS : OR
# KERNEL: ASSERT PASS : OR
# KERNEL:
# KERNEL: TC5 : XOR
# KERNEL: PASS : XOR
# KERNEL: ASSERT PASS : XOR
# KERNEL:
# KERNEL: TC6 : SLL
# KERNEL: PASS : SLL
# KERNEL: ASSERT PASS : SLL
# KERNEL:
# KERNEL: TC7 : SRL
# KERNEL: PASS : SRL
# KERNEL: ASSERT PASS : SRL
# KERNEL:
# KERNEL: TC8 : SRA
# KERNEL: PASS : SRA
# KERNEL: ASSERT PASS : SRA
# KERNEL:
# KERNEL: TC9 : SLT
# KERNEL: PASS : SLT
# KERNEL: ASSERT PASS : SLT
# KERNEL:
# KERNEL: TC10 : SLTU
# KERNEL: PASS : SLTU
# KERNEL: ASSERT PASS : SLTU
# KERNEL:
```

```
78 # KERNEL:
79 # KERNEL: TC11 : LOAD
80 # KERNEL: PASS : LOAD
81 # KERNEL: ASSERT PASS : LOAD
82 # KERNEL:
83 # KERNEL: TC12 : BEQ
84 # KERNEL: PASS : BEQ
85 # KERNEL: ASSERT PASS : BEQ
86 # KERNEL:
87 # KERNEL: TC13 : BNE
88 # KERNEL: PASS : BNE
89 # KERNEL: ASSERT PASS : BNE
90 # KERNEL:
91 # KERNEL: TC14 : BLT
92 # KERNEL: PASS : BLT
93 # KERNEL: ASSERT PASS : BLT
94 # KERNEL:
95 # KERNEL: TC15 : BGE
96 # KERNEL: PASS : BGE
97 # KERNEL: ASSERT PASS : BGE
98 # KERNEL:
99 # KERNEL: TC16 : LUI
100 # KERNEL: PASS : LUI
101 # KERNEL: ASSERT PASS : LUI
102 # KERNEL:
103 # KERNEL: TC17 : JAL
104 # KERNEL: PASS : JAL
105 # KERNEL: ASSERT PASS : JAL
106 # KERNEL:
107 # KERNEL: TC18 : DEFAULT
108 # KERNEL: PASS : DEFAULT
109 # KERNEL: ASSERT PASS : DEFAULT
110 # KERNEL:
111 # KERNEL: TC19 : ADDI
112 # KERNEL: PASS : ADDI
113 # KERNEL: ASSERT PASS : ADDI
114 # KERNEL:
115 # KERNEL: TC20 : BLTU
116 # KERNEL: PASS : BLTU
117 # KERNEL: ASSERT PASS : BLTU
118 # KERNEL:
```

```

# MODULE - work.tb_alu
#
# SUMMARY
#
# =====
# | Coverage Type | Weight | Hits/Total |
# =====
# | Covergroup Coverage | 1 | 100.000% |
# |-----|-----|-----|
# | Types | | 1 / 1 |
# =====
#
# WEIGHTED AVERAGE: 100.000%
#
#
# COVERGROUP COVERAGE
#
# =====
# | Covergroup | Hits | Goal / | Status |
# | | | At Least | |
# =====
# | TYPE /tb_alu/alu_cg | 100.000% | 100.000% | Covered |
# =====
# | INSTANCE ALU_COVERAGE | 100.000% | 100.000% | Covered |
# |-----|-----|-----|
# | COVERPOINT ALU_COVERAGE::cp_operation | 100.000% | 100.000% | Covered |
# |-----|-----|-----|
# | bin add_bin | 1 | 1 | Covered |
# | bin sub_bin | 1 | 1 | Covered |
# | bin sll_bin | 1 | 1 | Covered |
# | bin srl_bin | 1 | 1 | Covered |
# | bin sra_bin | 1 | 1 | Covered |
# | bin slt_bin | 1 | 1 | Covered |
# | bin sltu_bin | 1 | 1 | Covered |
# | bin xor_bin | 1 | 1 | Covered |
# | bin or_bin | 1 | 1 | Covered |
# | bin and_bin | 1 | 1 | Covered |
# | bin addi_bin | 1 | 1 | Covered |
# | bin loadstore_bin | 1 | 1 | Covered |
# | bin beq_bin | 1 | 1 | Covered |
# | bin bne_bin | 1 | 1 | Covered |
# | bin blt_bin | 1 | 1 | Covered |
# | bin bge_bin | 1 | 1 | Covered |
# | bin bltu_bin | 1 | 1 | Covered |
# | bin bgeu_bin | 1 | 1 | Covered |
# | bin lui_bin | 1 | 1 | Covered |
# | bin jal_bin | 1 | 1 | Covered |
# | default bin default_bin | 1 | - | Occurred |
# =====
#
# exit
# KERNEL:
# KERNEL: Final Coverage = 100.00%

```

4 ALU Control Verification

Module Description

The ALU Control module generates the appropriate ALU operation code based on the instruction type (alu_op) and instruction fields (func3, func7). It acts as the interface between the Control Unit and the ALU.

Testcases Executed

TC No	Description	Result
TC1	Load/Store Operation Decode Verification	PASS
TC2	BEQ Decode Verification	PASS
TC3	BNE Decode Verification	PASS
TC4	BLT Decode Verification	PASS
TC5	BGE Decode Verification	PASS
TC6	ADD Decode Verification	PASS
TC7	SUB Decode Verification	PASS
TC8	SLL Decode Verification	PASS
TC9	XOR Decode Verification	PASS
TC10	SRA Decode Verification	PASS
TC11	ADDI Decode Verification	PASS
TC12	SLLI Decode Verification	PASS
TC13	SRLI Decode Verification	PASS
TC14	JAL Decode Verification	PASS
TC15	JALR Decode Verification	PASS
TC16	LUI Decode Verification	PASS
TC17	AUIPC Decode Verification	PASS

Assertions Used

Assertion	Purpose	Result
Load/Store Assertion	Verify ALU_ADD generation for load/store instructions	PASS
Branch Assertions	Verify BEQ, BNE, BLT and BGE decoding	PASS
R-Type Assertions	Verify ADD, SUB, SLL and XOR decoding	PASS

Assertion	Purpose	Result
Shift Assertions	Verify SRA and SRLI decoding	PASS
I-Type Assertions	Verify ADDI and SLLI decoding	PASS
Jump Assertions	Verify JAL and JALR decoding	PASS
LUI Assertion	Verify LUI operation decoding	PASS
AUIPC Assertion	Verify AUIPC operation decoding	PASS

Verification Results

```

39 # KERNEL: TC1 : LOAD STORE
40 # KERNEL: PASS : TC1
41 # KERNEL: ASSERT PASS : LOAD STORE
42 # KERNEL:
43 # KERNEL: TC2 : BEQ
44 # KERNEL: PASS : TC2
45 # KERNEL: ASSERT PASS : BEQ
46 # KERNEL:
47 # KERNEL: TC3 : BNE
48 # KERNEL: PASS : TC3
49 # KERNEL: ASSERT PASS : BNE
50 # KERNEL:
51 # KERNEL: TC4 : BLT
52 # KERNEL: PASS : TC4
53 # KERNEL: ASSERT PASS : BLT
54 # KERNEL:
55 # KERNEL: TC5 : BGE
56 # KERNEL: PASS : TC5
57 # KERNEL: ASSERT PASS : BGE
58 # KERNEL:
59 # KERNEL: TC6 : ADD
60 # KERNEL: PASS : TC6
61 # KERNEL: ASSERT PASS : ADD
62 # KERNEL:
63 # KERNEL: TC7 : SUB
64 # KERNEL: PASS : TC7
65 # KERNEL: ASSERT PASS : SUB
66 # KERNEL:
67 # KERNEL: TC8 : SLL
68 # KERNEL: PASS : TC8
69 # KERNEL: ASSERT PASS : SLL
70 # KERNEL:
71 # KERNEL: TC9 : XOR
72 # KERNEL: PASS : TC9
73 # KERNEL: ASSERT PASS : XOR
74 # KERNEL:
75 # KERNEL: TC10 : SRA
76 # KERNEL: PASS : TC10
77 # KERNEL: ASSERT PASS : SRA
78 # KERNEL:

```

```

# KERNEL: TC11 : ADDI
# KERNEL: PASS : TC11
# KERNEL: ASSERT PASS : ADDI
# KERNEL:
# KERNEL: TC12 : SLLI
# KERNEL: PASS : TC12
# KERNEL: ASSERT PASS : SLLI
# KERNEL:
# KERNEL: TC13 : SRLI
# KERNEL: PASS : TC13
# KERNEL: ASSERT PASS : SRLI
# KERNEL:
# KERNEL: TC14 : JAL
# KERNEL: PASS : TC14
# KERNEL: ASSERT PASS : JAL
# KERNEL:
# KERNEL: TC15 : JALR
# KERNEL: PASS : TC15
# KERNEL: ASSERT PASS : JALR
# KERNEL:
# KERNEL: TC16 : LUI
# KERNEL: PASS : TC16
# KERNEL: ASSERT PASS : LUI
# KERNEL:
# KERNEL: TC17 : AUIPC
# KERNEL: PASS : TC17
# KERNEL: ASSERT PASS : AUIPC
# KERNEL:
# KERNEL: ALU_CONTROL VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%

```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_alu_control/alu_control_cg	100.000%	100.000%	Covered
INSTANCE ALU_CONTROL_COVERAGE	100.000%	100.000%	Covered
COVERPOINT ALU_CONTROL_COVERAGE::cp_aluop	100.000%	100.000%	Covered
bin load_store	1	1	Covered
bin branch	1	1	Covered
bin r_type	1	1	Covered
bin i_type	1	1	Covered
bin jal	1	1	Covered
bin jalr	1	1	Covered
bin lui	1	1	Covered
bin auipc	1	1	Covered

5. Adder Verification

Module Description

The Adder module generates the next sequential instruction address by adding a constant value of 4 to the current Program Counter (PC). It is used in the Instruction Fetch (IF) stage for normal program execution.

Testcases Executed

TC No	Description	Result
TC1	Zero Input Verification	PASS
TC2	Small Value Addition Verification	PASS
TC3	Medium Value Addition Verification	PASS
TC4	Large Value Addition Verification	PASS
TC5	Overflow Verification	PASS

Assertions Used

Assertion	Purpose	Result
Zero Value Assertion	Verify $PC = 0$ produces $PC+4 = 4$	PASS
Small Value Assertion	Verify correct addition for small values	PASS
Medium Value Assertion	Verify correct addition for medium values	PASS
Large Value Assertion	Verify correct addition for large values	PASS
Overflow Assertion	Verify wrap-around behavior at maximum address value	PASS

Verification Results

COVERGROUP COVERAGE				
=====				
Covergroup	Hits	Goal /	Status	
		At Least		
=====				
TYPE /tb_adder/adder_cg	100.000%	100.000%	Covered	
=====				
INSTANCE ADDER_COVERAGE	100.000%	100.000%	Covered	

COVERPOINT ADDER_COVERAGE::cp_pc	100.000%	100.000%	Covered	

bin zero_val	1	1	Covered	
bin small_val	1	1	Covered	
bin medium_val	1	1	Covered	
bin large_val	1	1	Covered	
bin max_val	1	1	Covered	
=====				

```
# KERNEL:
# KERNEL: TC1 : ZERO
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : ZERO
# KERNEL:
# KERNEL: TC2 : SMALL VALUE
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : SMALL
# KERNEL:
# KERNEL: TC3 : MEDIUM VALUE
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : MEDIUM
# KERNEL:
# KERNEL: TC4 : LARGE VALUE
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : LARGE
# KERNEL:
# KERNEL: TC5 : MAX VALUE
# KERNEL: PASS : TC5
# KERNEL: ASSERT PASS : OVERFLOW
# KERNEL:
# KERNEL: ADDER VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

6. Branch Decision Verification

Module Description

The Branch Decision module determines whether the Program Counter should be updated with a branch/jump target address. It generates the pc_src signal for PC selection and the flush signal for pipeline flushing during taken branches and jumps.

Testcases Executed

TC No	Description	Result
TC1	No Branch, No Jump Verification	PASS
TC2	Branch Condition High, Branch Disabled	PASS
TC3	Branch Enabled, Condition False	PASS
TC4	Branch Taken Verification	PASS
TC5	Jump Verification	PASS
TC6	Jump with Branch Condition High	PASS
TC7	Jump with Branch Enabled	PASS
TC8	Branch Taken + Jump Simultaneously	PASS

Assertions Used

Assertion	Purpose	Result
Idle Assertion	Verify pc_src and flush remain low when no branch/jump occurs	PASS
Branch Not Taken Assertion	Verify branch request without condition does not redirect PC	PASS
Branch Taken Assertion	Verify pc_src and flush are asserted for a taken branch	PASS
Jump Assertion	Verify pc_src and flush are asserted during jump instructions	PASS

Verification Results

```
# KERNEL :  
# KERNEL : TC1  
# KERNEL : ASSERT PASS : IDLE  
# KERNEL : PASS : TC1  
# KERNEL :  
# KERNEL : TC2  
# KERNEL : ASSERT PASS : IDLE  
# KERNEL : PASS : TC2  
# KERNEL :  
# KERNEL : TC3  
# KERNEL : ASSERT PASS : BRANCH NOT TAKEN  
# KERNEL : PASS : TC3  
# KERNEL :  
# KERNEL : TC4  
# KERNEL : ASSERT PASS : BRANCH TAKEN  
# KERNEL : PASS : TC4  
# KERNEL :  
# KERNEL : TC5  
# KERNEL : ASSERT PASS : JUMP  
# KERNEL : PASS : TC5  
# KERNEL :  
# KERNEL : TC6  
# KERNEL : ASSERT PASS : JUMP  
# KERNEL : PASS : TC6  
# KERNEL :  
# KERNEL : TC7  
# KERNEL : ASSERT PASS : JUMP  
# KERNEL : PASS : TC7  
# KERNEL :  
# KERNEL : TC8  
# KERNEL : ASSERT PASS : JUMP  
# KERNEL : PASS : TC8  
# KERNEL :  
# KERNEL : BRANCH_DECISION VERIFICATION COMPLETE  
# KERNEL :
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_branch_decision/branch_decision_cg	100.000%	100.000%	Covered
INSTANCE BRANCH_DECISION_COVERAGE	100.000%	100.000%	Covered
COVERPOINT BRANCH_DECISION_COVERAGE::cp_branch_condn	100.000%	100.000%	Covered
bin zero	4	1	Covered
bin one	4	1	Covered
COVERPOINT BRANCH_DECISION_COVERAGE::cp_branch	100.000%	100.000%	Covered
bin zero	4	1	Covered
bin one	4	1	Covered
COVERPOINT BRANCH_DECISION_COVERAGE::cp_jump	100.000%	100.000%	Covered
bin zero	4	1	Covered
bin one	4	1	Covered
COVERPOINT BRANCH_DECISION_COVERAGE::cp_pcsrc	100.000%	100.000%	Covered
bin zero	3	1	Covered
bin one	5	1	Covered
CROSS BRANCH_DECISION_COVERAGE::cross_all	100.000%	100.000%	Covered
bin <zero,zero,zero>	1	1	Covered
bin <zero,zero,one>	1	1	Covered
bin <zero,one,zero>	1	1	Covered
bin <zero,one,one>	1	1	Covered
bin <one,zero,zero>	1	1	Covered
bin <one,zero,one>	1	1	Covered
bin <one,one,zero>	1	1	Covered
bin <one,one,one>	1	1	Covered

7. Control Unit Verification

Module Description

The Control Unit decodes the instruction opcode and generates the required control signals for different RISC-V instruction types such as R-Type, I-Type, Load, Store, Branch, JAL, JALR, LUI, and AUIPC. It also generates the result_src signal for write-back path selection.

Testcases Executed

TC No	Description	Result
TC1	R-Type Instruction Decode	PASS
TC2	I-Type Instruction Decode	PASS
TC3	Load Instruction Decode	PASS
TC4	Store Instruction Decode	PASS
TC5	Branch Instruction Decode	PASS
TC6	JAL Instruction Decode	PASS
TC7	JALR Instruction Decode	PASS
TC8	LUI Instruction Decode	PASS
TC9	AUIPC Instruction Decode	PASS
TC10	Invalid Opcode Default Case	PASS

Assertions Used

Assertion	Purpose	Result
R-Type Assertion	Verify correct control signals for R-Type instructions	PASS
I-Type Assertion	Verify correct control signals for I-Type instructions	PASS
Load Assertion	Verify Load control path generation	PASS
Store Assertion	Verify Store control path generation	PASS
Branch Assertion	Verify Branch control signal generation	PASS
JAL Assertion	Verify Jump and Link control signals	PASS
JALR Assertion	Verify Jump and Link Register control signals	PASS
LUI Assertion	Verify LUI operation decode	PASS
AUIPC Assertion	Verify AUIPC operation decode	PASS
Default Assertion	Verify safe default outputs for invalid opcode	PASS

Verification Results

```
39 # KERNEL: TC1 : R_TYPE
40 # KERNEL: PASS : TC1
41 # KERNEL: ASSERT PASS : R_TYPE
42 # KERNEL:
43 # KERNEL: TC2 : I_TYPE
44 # KERNEL: PASS : TC2
45 # KERNEL: ASSERT PASS : I_TYPE
46 # KERNEL:
47 # KERNEL: TC3 : LOAD
48 # KERNEL: PASS : TC3
49 # KERNEL: ASSERT PASS : LOAD
50 # KERNEL:
51 # KERNEL: TC4 : STORE
52 # KERNEL: PASS : TC4
53 # KERNEL: ASSERT PASS : STORE
54 # KERNEL:
55 # KERNEL: TC5 : BRANCH
56 # KERNEL: PASS : TC5
57 # KERNEL: ASSERT PASS : BRANCH
58 # KERNEL:
59 # KERNEL: TC6 : JAL
60 # KERNEL: PASS : TC6
61 # KERNEL: ASSERT PASS : JAL
62 # KERNEL:
63 # KERNEL: TC7 : JALR
64 # KERNEL: PASS : TC7
65 # KERNEL: ASSERT PASS : JALR
66 # KERNEL:
67 # KERNEL: TC8 : LUI
68 # KERNEL: PASS : TC8
69 # KERNEL: ASSERT PASS : LUI
70 # KERNEL:
71 # KERNEL: TC9 : AUIPC
72 # KERNEL: PASS : TC9
73 # KERNEL: ASSERT PASS : AUIPC
74 # KERNEL:
75 # KERNEL: TC10 : INVALID
76 # KERNEL: PASS : TC10
77 # KERNEL: ASSERT PASS : DEFAULT
78 # KERNEL:
```

Final Coverage = 100.00%

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_control/control_cg	100.000%	100.000%	Covered
INSTANCE CONTROL_COVERAGE	100.000%	100.000%	Covered
COVERPOINT CONTROL_COVERAGE::cp_opcode	100.000%	100.000%	Covered
bin r_type	1	1	Covered
bin i_type	1	1	Covered
bin load	1	1	Covered
bin store	1	1	Covered
bin branch	1	1	Covered
bin jal	1	1	Covered
bin jalr	1	1	Covered
bin lui	1	1	Covered
bin auipc	1	1	Covered
default bin invalid	1	-	Occurred

8 Control Mux Verification

Module Description

The Control Mux module is used for hazard handling in the pipeline. During normal operation, it forwards all control signals to the next stage. During a stall condition (control_mux_sel = 1), it inserts a NOP by forcing all control outputs to zero.

Testcases Executed

TC No	Description	Result
TC1	Normal Flow Verification	PASS
TC2	Stall / NOP Insertion Verification	PASS
TC3	Dynamic Switching Verification	PASS

Assertions Used

Assertion	Purpose	Result
Normal Flow Assertion	Verify all control signals pass correctly when stall is inactive	PASS
Stall NOP Assertion	Verify all outputs are cleared during stall condition	PASS
Dynamic Switching Assertion	Verify correct transition between normal and stall operation	PASS

Verification Result

```
# KERNEL:
# KERNEL: TC1 : NORMAL FLOW
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : NORMAL FLOW
# KERNEL:
# KERNEL: TC2 : STALL NOP
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : STALL NOP
# KERNEL:
# KERNEL: TC3 : DYNAMIC SWITCHING
# KERNEL: ASSERT PASS : NORMAL
# KERNEL: ASSERT PASS : STALL
# KERNEL: PASS : TC3
# KERNEL:
# KERNEL: CONTROL_MUX VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

MODULE - work.tb_control_mux

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

#

SUMMARY

Coverage Type	Weight	Hits/Total
Covergroup Coverage	1	100.000%
Types		1 / 1

WEIGHTED AVERAGE: 100.000%

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_control_mux/control_mux_cg	100.000%	100.000%	Covered
INSTANCE CONTROL_MUX_COVERAGE	100.000%	100.000%	Covered
COVERPOINT CONTROL_MUX_COVERAGE::cp_sel	100.000%	100.000%	Covered
bin normal_flow	2	1	Covered
bin stall_nop	2	1	Covered

exit

KERNEL:

KERNEL: Final Coverage = 100.00%

VSIM: Simulation has finished.

9. Data Memory Verification

Module Description

The Data Memory module provides read and write access to a 16 KB memory array. It supports synchronous write operations, synchronous reset functionality, and combinational read operations for load/store instructions.

Testcases Executed

TC No	Description	Result
TC1	Basic Read/Write Verification	PASS
TC2	Reset Clears Memory Verification	PASS
TC3	Read Disable Verification	PASS
TC4	Multiple Address Read/Write Verification	PASS
TC5	Boundary Address Verification	PASS
TC6	Mid Address Access Verification	PASS
TC7	High Address Access Verification	PASS
TC8	Simultaneous Read and Write Verification	PASS
TC9	Cross Coverage Completion	PASS
TC10	Low Address with Write Disabled	PASS
TC11	Mid Address Bin Completion	PASS

Assertions Used

Assertion	Purpose	Result
Reset Assertion	Verify memory locations are cleared after reset	PASS
Write Assertion	Verify data is written correctly to addressed memory location	PASS
Read Disable Assertion	Verify output remains zero when mem_read is deasserted	PASS
Read-After-Write Assertion	Verify correct data is returned after a write operation	PASS
No Unknown Assertion	Verify output never contains X/Z values	PASS

Functional Coverage

Coverpoint	Bins Covered
Reset Coverage	Asserted, Deasserted
Read Enable Coverage	ON, OFF
Write Enable Coverage	ON, OFF
Address Coverage	Low, Mid, High, Boundary Addresses
Data Coverage	Low, Mid, High Data Values
Read/Write Cross Coverage	All Combinations
Address vs Write Cross Coverage All Address Regions Exercised	

```

KERNEL: ASDB file was created in location /home/run
run -all

KERNEL: Starting Data Memory Verification
KERNEL: WRITE: MEM[1] = 100
KERNEL: PASS : Read/Write Test
KERNEL:
KERNEL: TC2 : Reset Verification
KERNEL: PASS : Reset Cleared Memory
KERNEL:
KERNEL: TC3 : Read Disable Verification
KERNEL: WRITE: MEM[2] = 200
KERNEL: PASS : Read Disable
KERNEL:
KERNEL: TC4 : Multiple Address Verification
KERNEL: WRITE: MEM[1] = 111
KERNEL: WRITE: MEM[2] = 222
KERNEL: WRITE: MEM[3] = 333
KERNEL: PASS : Address 4
KERNEL: PASS : Address 8
KERNEL: PASS : Address 12
KERNEL:
KERNEL: TC5 : Boundary Address Verification
KERNEL: WRITE: MEM[fff] = 9999
KERNEL: PASS : Boundary Address

```

```

KERNEL: TC6 : Mid Address Coverage
KERNEL: WRITE: MEM[400] = 555
KERNEL: PASS : Mid Address
KERNEL:
KERNEL: TC7 : High Address Coverage
KERNEL: WRITE: MEM[800] = 777
KERNEL: PASS : High Address
KERNEL:
KERNEL: TC8 : Read + Write Simultaneously
KERNEL: WRITE: MEM[4] = 1234
KERNEL:
KERNEL: TC9 : Cross Coverage Completion
KERNEL:
KERNEL: TC10 : Low Address + Write OFF
KERNEL:
KERNEL: TC11 : Mid Address Bin Completion
KERNEL: WRITE: MEM[40] = 444
KERNEL: PASS : Mid Address Bin
KERNEL:
KERNEL: Coverage = 100.00%

```

COVERGROUP COVERAGE				
=====				
Covergroup	Hits	Goal /	Status	
		At Least		
=====				
TYPE /tb_data_memory/mem_cg	100.000%	100.000%	Covered	
=====				
INSTANCE	100.000%	100.000%	Covered	
----- ----- ----- -----				
COVERPOINT ::cp_rst	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin asserted	2	1	Covered	
bin deasserted	26	1	Covered	
----- ----- ----- -----				
COVERPOINT ::cp_read	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin off	11	1	Covered	
bin on	17	1	Covered	
----- ----- ----- -----				
COVERPOINT ::cp_write	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin off	18	1	Covered	
bin on	10	1	Covered	
----- ----- ----- -----				
COVERPOINT ::cp_addr	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin low_addr	17	1	Covered	
bin mid_addr	2	1	Covered	
bin high_addr	5	1	Covered	
bin boundary	4	1	Covered	
----- ----- ----- -----				

----- ----- ----- -----				
COVERPOINT ::cp_data	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin low_data	11	1	Covered	
bin mid_data	9	1	Covered	
bin high_data	8	1	Covered	
----- ----- ----- -----				
CROSS ::cross_rw	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin <off,off>	6	1	Covered	
bin <off,on>	5	1	Covered	
bin <on,off>	12	1	Covered	
bin <on,on>	5	1	Covered	
----- ----- ----- -----				
CROSS ::cross_addr_write	100.000%	100.000%	Covered	
----- ----- ----- -----				
bin <low_addr,off>	11	1	Covered	
bin <low_addr,on>	6	1	Covered	
bin <mid_addr,off>	1	1	Covered	
bin <mid_addr,on>	1	1	Covered	
bin <high_addr,off>	3	1	Covered	
bin <high_addr,on>	2	1	Covered	
bin <boundary,off>	3	1	Covered	
bin <boundary,on>	1	1	Covered	
----- ----- ----- -----				

10. EX/MEM Pipeline Register Verification

Module Description

The EX/MEM pipeline register transfers data signals, control signals, PC+4 value, and result source information from the Execute stage to the Memory stage. It also supports synchronous reset functionality.

Testcases Executed

TC No	Description	Result
TC1	Reset Verification	PASS
TC2	Data Path Transfer Verification	PASS
TC3	Control Signal Transfer Verification	PASS
TC4	PC+4 Transfer Verification	PASS
TC5	Result Source (PC+4 Path) Verification	PASS
TC6	Result Source (Memory Path) Verification	PASS
TC7	Result Source (Default Path) Verification	PASS
TC8	Mid Register Address Verification	PASS
TC9	High Register Address Verification	PASS
TC10	Memory Path + RegWrite Enabled	PASS
TC11	Memory Path + RegWrite Disabled	PASS
TC12	Default Path + RegWrite Enabled	PASS
TC13	Default Path + RegWrite Disabled	PASS
TC14	PC+4 Path + RegWrite Disabled	PASS

Assertions Used

Assertion	Purpose	Result
Reset Assertion	Verify all outputs are cleared during reset	PASS
Data Path Assertion	Verify ALU result, write data and destination register transfer correctly	PASS
Control Path Assertion	Verify control signals transfer correctly	PASS
PC+4 Assertion	Verify PC+4 value propagates correctly to MEM stage	PASS
Result Source Assertion	Verify result source information transfers correctly	PASS
Register Transfer Assertion	Verify destination register values transfer correctly	PASS
RegWrite Cross Assertion	Verify result source and reg_write combinations operate correctly	PASS

Functional Coverage

Coverpoint	Bins Covered
Reset Coverage	Asserted, Deasserted
Memory Read Coverage	ON, OFF
Memory Write Coverage	ON, OFF
Memory-to-Reg Coverage	ON, OFF
Register Write Coverage	ON, OFF
Result Source Coverage	ALU Path, Memory Path, PC+4 Path, Default Path
Destination Register Coverage	x0, Low Registers, Mid Registers, High Registers
Cross Coverage	Result Source $\times$ Register Write

Verification Results

```
# KERNEL: TC1 : Reset Verification
# KERNEL: PASS : Reset Verification
# KERNEL:
# KERNEL: TC2 : Data Path Transfer
# KERNEL: PASS : Data Path Transfer
# KERNEL: ASSERT PASS : DATA PATH
# KERNEL:
# KERNEL: TC3 : Control Signal Transfer
# KERNEL: PASS : Control Signal Transfer
# KERNEL: ASSERT PASS : CONTROL PATH
# KERNEL:
# KERNEL: TC4 : PC+4 Transfer
# KERNEL: PASS : PC+4 Transfer
# KERNEL: ASSERT PASS : PC+4
# KERNEL:
# KERNEL: TC5 : result_src Transfer
# KERNEL: PASS : result_src Transfer
# KERNEL: ASSERT PASS : RESULT_SRC
# KERNEL:
# KERNEL: TC6 : result_src = MEM
# KERNEL: ASSERT PASS : RESULT_SRC_MEM
# KERNEL:
# KERNEL: TC7 : result_src = DEFAULT
# KERNEL: ASSERT PASS : RESULT_SRC_DEFAULT
# KERNEL:
# KERNEL: TC8 : Mid Register
# KERNEL: ASSERT PASS : MID_REGISTER
# KERNEL:
# KERNEL: TC9 : High Register
# KERNEL: ASSERT PASS : HIGH_REGISTER
# KERNEL: ASSERT PASS : MEM_PATH_REGWRITE_ON
# KERNEL: ASSERT PASS : MEM_PATH_REGWRITE_OFF
# KERNEL: ASSERT PASS : DEFAULT_PATH_REGWRITE_ON
# KERNEL: ASSERT PASS : DEFAULT_PATH_REGWRITE_OFF
# KERNEL:
# KERNEL: TC14 : PC4 Path + RegWrite OFF
# KERNEL: ASSERT PASS : PC4_PATH_REGWRITE_OFF
# KERNEL:
# KERNEL: EX_MEM VERIFICATION COMPLETE
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_ex_mem/ex_mem_cg	100.000%	100.000%	Covered
INSTANCE EX_MEM_COVERAGE	100.000%	100.000%	Covered
COVERPOINT EX_MEM_COVERAGE::cp_rst	100.000%	100.000%	Covered
bin asserted	1	1	Covered
bin deasserted	14	1	Covered
COVERPOINT EX_MEM_COVERAGE::cp_mem_read	100.000%	100.000%	Covered
bin off	2	1	Covered
bin on	13	1	Covered
COVERPOINT EX_MEM_COVERAGE::cp_mem_write	100.000%	100.000%	Covered
bin off	2	1	Covered
bin on	13	1	Covered
COVERPOINT EX_MEM_COVERAGE::cp_mem_to_reg	100.000%	100.000%	Covered
bin off	2	1	Covered
bin on	13	1	Covered
COVERPOINT EX_MEM_COVERAGE::cp_reg_write	100.000%	100.000%	Covered
bin off	6	1	Covered
bin on	9	1	Covered
COVERPOINT EX_MEM_COVERAGE::cp_result_src	100.000%	100.000%	Covered
bin alu_path	4	1	Covered
bin mem_path	3	1	Covered
bin pc4_path	3	1	Covered
bin default_path	5	1	Covered

COVERPOINT EX_MEM_COVERAGE::cp_rd	100.000%	100.000%	Covered
bin x0	1	1	Covered
bin low_reg	6	1	Covered
bin mid_reg	1	1	Covered
bin high_reg	7	1	Covered
CROSS EX_MEM_COVERAGE::cross_result_regwrite	100.000%	100.000%	Covered
bin <alu_path,off>	2	1	Covered
bin <alu_path,on>	2	1	Covered
bin <mem_path,off>	1	1	Covered
bin <mem_path,on>	2	1	Covered
bin <pc4_path,off>	2	1	Covered
bin <pc4_path,on>	1	1	Covered
bin <default_path,off>	1	1	Covered
bin <default_path,on>	4	1	Covered

11. Forwarding Unit Verification

Module Description

The Forwarding Unit resolves data hazards in the pipelined RISC-V processor by forwarding data from the EX/MEM and MEM/WB pipeline stages to the ID/EX stage. It prioritizes EX/MEM forwarding over MEM/WB forwarding and prevents forwarding from register x0.

Testcases Executed

TC No	Description	Result
TC1	No Forwarding Required	PASS
TC2	EX Stage Forwarding for Source A	PASS
TC3	EX Stage Forwarding for Source B	PASS
TC4	MEM/WB Stage Forwarding for Source A	PASS
TC5	MEM/WB Stage Forwarding for Source B	PASS
TC6	EX/MEM Priority Verification for Source A	PASS
TC7	EX/MEM Priority Verification for Source B	PASS
TC8	Zero Register (x0) Forwarding Check	PASS

Assertions Used

Assertion	Purpose	Result
No Forwarding Assertion	Verify forwarding outputs remain 00 when no dependency exists	PASS
EX Forward A Assertion	Verify Source A receives data from EX/MEM stage	PASS
EX Forward B Assertion	Verify Source B receives data from EX/MEM stage	PASS
MEM Forward A Assertion	Verify Source A receives data from MEM/WB stage	PASS
MEM Forward B Assertion	Verify Source B receives data from MEM/WB stage	PASS
Priority Assertion	Verify EX/MEM forwarding has higher priority than MEM/WB	PASS
Zero Register Assertion	Verify forwarding is disabled for register x0	PASS

Functional Coverage

Coverpoint	Bins Covered
Forward_A Coverage	No Forwarding, EX Forwarding, MEM Forwarding
Forward_B Coverage	No Forwarding, EX Forwarding, MEM Forwarding

Verification result

```
# KERNEL:
# KERNEL: TC1 : NO FORWARDING
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : NO FORWARDING
# KERNEL:
# KERNEL: TC2 : EX FORWARD A
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : EX FORWARD A
# KERNEL:
# KERNEL: TC3 : EX FORWARD B
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : EX FORWARD B
# KERNEL:
# KERNEL: TC4 : MEM FORWARD A
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : MEM FORWARD A
# KERNEL:
# KERNEL: TC5 : MEM FORWARD B
# KERNEL: PASS : TC5
# KERNEL: ASSERT PASS : MEM FORWARD B
# KERNEL:
# KERNEL: TC6 : PRIORITY CHECK A
# KERNEL: PASS : TC6
# KERNEL: ASSERT PASS : EX PRIORITY
# KERNEL:
# KERNEL: TC7 : PRIORITY CHECK B
# KERNEL: PASS : TC7
# KERNEL: ASSERT PASS : EX PRIORITY
# KERNEL:
# KERNEL: TC8 : ZERO REGISTER
# KERNEL: PASS : TC8
# KERNEL: ASSERT PASS : ZERO REGISTER
# KERNEL:
# KERNEL: FORWARDING VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

Final Coverage = 100.00%

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_forwarding/forwarding_cg	100.000%	100.000%	Covered
INSTANCE FORWARDING_COVERAGE	100.000%	100.000%	Covered
COVERPOINT FORWARDING_COVERAGE::cp_forward_A	100.000%	100.000%	Covered
bin no_forward	5	1	Covered
bin ex_forward	2	1	Covered
bin mem_forward	1	1	Covered
COVERPOINT FORWARDING_COVERAGE::cp_forward_B	100.000%	100.000%	Covered
bin no_forward	5	1	Covered
bin ex_forward	2	1	Covered
bin mem_forward	1	1	Covered

12. Hazard Detection Unit Verification

Module Description

The Hazard Detection Unit detects load-use data hazards in the pipelined RISC-V processor. When a hazard is detected, it stalls the pipeline by disabling PC updates, preventing IF/ID and ID/EX register updates, and inserting a NOP through the control mux.

Testcases Executed

TC No	Description	Result
TC1	No Hazard Condition	PASS
TC2	RS1 Load-Use Hazard Detection	PASS
TC3	RS2 Load-Use Hazard Detection	PASS
TC4	MemRead Disabled Condition	PASS
TC5	Zero Register Check	PASS
TC6	Double Hazard Detection (RS1 & RS2 Match)	PASS
TC7	Dynamic Hazard Detection and Recovery	PASS

Assertions Used

Assertion	Purpose	Result
No Hazard Assertion	Verify normal pipeline operation when no hazard exists	PASS
RS1 Hazard Assertion	Verify pipeline stall when RS1 matches destination register	PASS
RS2 Hazard Assertion	Verify pipeline stall when RS2 matches destination register	PASS
MemRead Disable Assertion	Verify no stall occurs when MemRead is disabled	PASS
Zero Register Assertion	Verify hazards are ignored for register x0	PASS
Double Hazard Assertion	Verify stall when both source registers match destination register	PASS
Dynamic Switching Assertion	Verify hazard detection, stall, and recovery transitions correctly	PASS

Functional Coverage

Coverpoint	Bins Covered
PC_Write Coverage	Stall, Normal Operation
Control Mux Coverage	NOP Inserted, Normal Flow

Verification Results

```
# KERNEL:
# KERNEL: TC1 : NO HAZARD
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : NO HAZARD
# KERNEL:
# KERNEL: TC2 : RS1 HAZARD
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : RS1 HAZARD
# KERNEL:
# KERNEL: TC3 : RS2 HAZARD
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : RS2 HAZARD
# KERNEL:
# KERNEL: TC4 : MEMREAD DISABLED
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : MEMREAD DISABLED
# KERNEL:
# KERNEL: TC5 : ZERO REGISTER
# KERNEL: PASS : TC5
# KERNEL: ASSERT PASS : ZERO REGISTER
# KERNEL:
# KERNEL: TC6 : DOUBLE HAZARD
# KERNEL: PASS : TC6
# KERNEL: ASSERT PASS : DOUBLE HAZARD
# KERNEL:
# KERNEL: TC7 : DYNAMIC SWITCHING
# KERNEL: ASSERT PASS : NORMAL
# KERNEL: ASSERT PASS : HAZARD
# KERNEL: ASSERT PASS : RECOVERY
# KERNEL:
# KERNEL: HAZARD DETECTION VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_hazard_detection/hazard_cg	100.000%	100.000%	Covered
INSTANCE HAZARD_COVERAGE	100.000%	100.000%	Covered
COVERPOINT HAZARD_COVERAGE::cp_pc_write	100.000%	100.000%	Covered
bin stall	3	1	Covered
bin normal	4	1	Covered
COVERPOINT HAZARD_COVERAGE::cp_control_mux	100.000%	100.000%	Covered
bin nop_inserted	3	1	Covered
bin normal_flow	4	1	Covered

13. ID/EX Pipeline Register Verification

Module Description

The ID/EX pipeline register transfers instruction data, register values, immediate values, control signals, PC+4 information, and result source selection from the Instruction Decode (ID) stage to the Execute (EX) stage. It also supports reset, flush, and stall operations for hazard handling and pipeline control.

Testcases Executed

TC No	Description	Result
TC1	Reset Verification	PASS
TC2	Normal Data and Control Signal Transfer	PASS
TC3	Stall Verification	PASS
TC4	Flush Verification	PASS
TC5	Flush Priority over Stall Verification	PASS
TC6	Result Source = Memory Path	PASS
TC7	Result Source = PC+4 Path	PASS
TC8	Second Normal Transfer Verification	PASS

Assertions Used

Assertion	Purpose	Result
Reset Assertion	Verify all outputs are cleared during reset	PASS
Flush Assertion	Verify NOP insertion and output clearing during flush	PASS
Stall Assertion	Verify control signals are cleared during stall condition	PASS
Transfer Assertion	Verify all data and control signals transfer correctly during normal operation	PASS

Functional Coverage

Coverpoint	Bins Covered
Reset Coverage	Reset Asserted, Reset Deasserted
Flush Coverage	Flush Active, Flush Inactive
Stall Coverage	Stall Active, Stall Inactive
Register Write Coverage	Enabled, Disabled
Result Source Coverage	ALU Path, Memory Path, PC+4 Path

Verification Results

```
# KERNEL: TC1 : RESET
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : STALL
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL: PASS : TC1
# KERNEL:
# KERNEL: TC2 : NORMAL TRANSFER
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : STALL
# KERNEL: PASS : TC2
# KERNEL:
# KERNEL: TC3 : STALL
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL: PASS : TC3
# KERNEL:
# KERNEL: TC4 : FLUSH
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : STALL
# KERNEL: ASSERT PASS : STALL
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL: PASS : TC4
# KERNEL:
# KERNEL: TC5 : FLUSH PRIORITY
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: TC6 : MEM PATH
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : STALL
# KERNEL: ASSERT PASS : STALL
# KERNEL: PASS : TC6
# KERNEL:
# KERNEL: TC7 : PC+4 PATH
# KERNEL: ASSERT PASS : RESET
# KERNEL: ASSERT PASS : FLUSH
# KERNEL: ASSERT PASS : STALL
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL: PASS : TC7
# KERNEL:
# KERNEL: TC8 : SECOND TRANSFER
# KERNEL: ASSERT PASS : RESET
```

```

# COVERGROUP COVERAGE
# =====
# | Covergroup | Hits | Goal / | Status |
# | | | At Least | |
# =====
# | TYPE /tb_ID_EX/id_ex_cg | 100.000% | 100.000% | Covered |
# =====
# | INSTANCE ID_EX_COVERAGE | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | COVERPOINT ID_EX_COVERAGE::cp_rst | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | bin reset_on | 1 | 1 | Covered |
# | bin reset_off | 8 | 1 | Covered |
# |-----|-----|-----|-----|
# | COVERPOINT ID_EX_COVERAGE::cp_flush | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | bin flush_on | 2 | 1 | Covered |
# | bin flush_off | 7 | 1 | Covered |
# |-----|-----|-----|-----|
# | COVERPOINT ID_EX_COVERAGE::cp_stall | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | bin stall_on | 2 | 1 | Covered |
# | bin stall_off | 7 | 1 | Covered |
# |-----|-----|-----|-----|
# | COVERPOINT ID_EX_COVERAGE::cp_regwrite | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | bin enabled | 4 | 1 | Covered |
# | bin disabled | 4 | 1 | Covered |
# |-----|-----|-----|-----|
# | COVERPOINT ID_EX_COVERAGE::cp_result_src | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | bin alu_path | 5 | 1 | Covered |
# | bin mem_path | 2 | 1 | Covered |
# | bin pc4_path | 1 | 1 | Covered |
# =====
#
# exit
# KERNEL:
# KERNEL: Final Coverage = 100.00%
# VSIM: Simulation has finished.

```

14. IF/ID Pipeline Register Verification

Module Description

The IF/ID pipeline register transfers instruction, PC, and PC+4 information from the Instruction Fetch (IF) stage to the Instruction Decode (ID) stage. It supports reset, pipeline stalling through IF_ID_write, and flushing for branch/jump handling. The module retains previous values during stall conditions and inserts a NOP during flush operations.

Testcases Executed

TC No	Description	Result
TC1	Reset Verification	PASS
TC2	Normal Data Transfer Verification	PASS
TC3	Stall Verification	PASS
TC4	Flush Verification	PASS
TC5	Flush Priority Over Write Verification	PASS
TC6	Normal Transfer After Flush Verification	PASS

Assertions Used

Assertion	Purpose	Result
Reset Assertion	Verify outputs are cleared during reset	PASS
Transfer Assertion	Verify instruction, PC, and PC+4 transfer correctly during normal operation	PASS
Stall Assertion	Verify previous values are retained during stall condition	PASS
Flush Assertion	Verify outputs are cleared during flush operation	PASS

Functional Coverage

Coverpoint	Bins Covered
Reset Coverage	Reset Asserted, Reset Deasserted
Flush Coverage	Flush Active, Flush Inactive
Write Control Coverage	Write Enabled, Write Disabled

Verification Result

```
# KERNEL:
# KERNEL: TC1 : RESET
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : RESET
# KERNEL:
# KERNEL: TC2 : NORMAL TRANSFER
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : TRANSFER
# KERNEL:
# KERNEL: TC3 : STALL
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : STALL
# KERNEL:
# KERNEL: TC4 : FLUSH
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : FLUSH
# KERNEL:
# KERNEL: TC5 : FLUSH PRIORITY
# KERNEL: PASS : TC5
# KERNEL: ASSERT PASS : FLUSH PRIORITY
# KERNEL:
# KERNEL: TC6 : NORMAL AGAIN
# KERNEL: PASS : TC6
# KERNEL: ASSERT PASS : NORMAL AGAIN
# KERNEL:
# KERNEL: IF_ID VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_IF_ID/ifid_cg	100.000%	100.000%	Covered
INSTANCE IF_ID_COVERAGE	100.000%	100.000%	Covered
COVERPOINT IF_ID_COVERAGE::cp_rst	100.000%	100.000%	Covered
bin reset_on	1	1	Covered
bin reset_off	6	1	Covered
COVERPOINT IF_ID_COVERAGE::cp_flush	100.000%	100.000%	Covered
bin flush_on	2	1	Covered
bin flush_off	5	1	Covered
COVERPOINT IF_ID_COVERAGE::cp_write	100.000%	100.000%	Covered
bin write_on	4	1	Covered
bin write_off	3	1	Covered

15. Immediate Generator (IMM_GEN) Verification

Module Description

The Immediate Generator (IMM_GEN) module extracts and sign-extends immediate values from RISC-V instructions. It supports I-Type, Load, JALR, Store, Branch, LUI, AUIPC, and JAL instruction formats and generates the corresponding 32-bit immediate value.

Testcases Executed

TC No	Description	Result
TC1	I-Type Immediate Generation	PASS
TC2	Load Immediate Generation	PASS
TC3	JALR Immediate Generation	PASS
TC4	Store Immediate Generation	PASS
TC5	Branch Immediate Generation	PASS
TC6	LUI Immediate Generation	PASS
TC7	AUIPC Immediate Generation	PASS
TC8	JAL Immediate Generation	PASS
TC9	Invalid Opcode Handling	PASS
TC10	Zero Immediate Verification	PASS
TC11	Medium Immediate Verification	PASS
TC12	Functional Coverage Bin Completion	PASS

Assertions Used

Assertion	Purpose	Result
I-Type Assertion	Verify correct I-Type immediate extraction	PASS
Load Assertion	Verify correct Load immediate extraction	PASS
JALR Assertion	Verify correct JALR immediate extraction	PASS
Store Assertion	Verify correct Store immediate extraction	PASS
Branch Assertion	Verify correct Branch immediate extraction	PASS
LUI Assertion	Verify correct LUI immediate generation	PASS
AUIPC Assertion	Verify correct AUIPC immediate generation	PASS
JAL Assertion	Verify correct JAL immediate generation	PASS
Invalid Opcode Assertion	Verify default immediate value generation	PASS

Assertion	Purpose	Result
Zero Immediate Assertion	Verify zero immediate handling	PASS

Functional Coverage

Coverpoint	Bins Covered
Opcode Coverage	I-Type, Load, JALR, Store, Branch, LUI, AUIPC, JAL, Invalid Opcode
Immediate Value Coverage	Zero, Small Positive, Medium Positive, Large Positive

```
# KERNEL: TC1 : I-TYPE
# KERNEL: ASSERT PASS : I_TYPE
# KERNEL: IMM = 100
# KERNEL:
# KERNEL: TC2 : LOAD
# KERNEL: ASSERT PASS : LOAD
# KERNEL: IMM = 32
# KERNEL:
# KERNEL: TC3 : JALR
# KERNEL: ASSERT PASS : JALR
# KERNEL: IMM = 16
# KERNEL:
# KERNEL: TC4 : STORE
# KERNEL: ASSERT PASS : STORE
# KERNEL: IMM = 100
# KERNEL:
# KERNEL: TC5 : BRANCH
# KERNEL: ASSERT PASS : BRANCH
# KERNEL: IMM = 16
# KERNEL:
# KERNEL: TC6 : LUI
# KERNEL: ASSERT PASS : LUI
# KERNEL: IMM = 305418240
# KERNEL:
# KERNEL: TC7 : AUIPC
# KERNEL: ASSERT PASS : AUIPC
# KERNEL: IMM = 305418240
# KERNEL:
# KERNEL: TC8 : JAL
# KERNEL: ASSERT PASS : JAL
# KERNEL: IMM = 32
# KERNEL:
# KERNEL: TC9 : INVALID
# KERNEL: ASSERT PASS : INVALID OPCODE
# KERNEL: IMM = 0
# KERNEL:
# KERNEL: TC10 : ZERO IMM
# KERNEL: ASSERT PASS : ZERO IMM
# KERNEL: IMM = 0
# KERNEL:
# KERNEL: TC11 : MEDIUM IMM
# KERNEL: ASSERT PASS : MEDIUM IMM
# KERNEL: IMM = 500
# KERNEL:
# KERNEL: TC12 : FORCE MEDIUM BIN
# KERNEL: ASSERT PASS : MEDIUM IMM
# KERNEL: IMM = 800
# KERNEL:
# KERNEL: IMM_GEN VERIFICATION COMPLETE
# KERNEL:
```

COVERGROUP COVERAGE			
Covergroup	Hits	Goal / At Least	Status
TYPE /tb_imm_gen/imm_gen_cg	100.000%	100.000%	Covered
INSTANCE IMM_GEN_COVERAGE	100.000%	100.000%	Covered
COVERPOINT IMM_GEN_COVERAGE::cp_opcode	100.000%	100.000%	Covered
bin i_type	4	1	Covered
bin load	1	1	Covered
bin jalr	1	1	Covered
bin store	1	1	Covered
bin branch	1	1	Covered
bin lui	1	1	Covered
bin auipc	1	1	Covered
bin jal	1	1	Covered
default bin invalid	1	-	Occurred
COVERPOINT IMM_GEN_COVERAGE::cp_imm	100.000%	100.000%	Covered
bin zero_val	2	1	Covered
bin small_pos	6	1	Covered
bin medium_pos	1	1	Covered
bin large_pos	2	1	Covered

16. Instruction Memory Verification

Module Description

The Instruction Memory module stores program instructions and provides instruction fetch functionality. It reads instructions from the program.mem file and outputs the instruction corresponding to the Program Counter (PC) address. The module supports word-aligned instruction access through pc[31:2].

Testcases Executed

TC No	Description	Result
TC1	Instruction Fetch at Address 0	PASS
TC2	Instruction Fetch at Address 1	PASS
TC3	Instruction Fetch at Address 2	PASS
TC4	Instruction Fetch at Address 3	PASS
TC5	Address Alignment Verification (PC=1)	PASS
TC6	Address Alignment Verification (PC=2)	PASS
TC7	Address Alignment Verification (PC=3)	PASS

Assertions Used

Assertion	Purpose	Result
Address 0 Assertion	Verify correct instruction fetched from address 0	PASS
Address 1 Assertion	Verify correct instruction fetched from address 1	PASS
Address 2 Assertion	Verify correct instruction fetched from address 2	PASS
Address 3 Assertion	Verify correct instruction fetched from address 3	PASS
Alignment Assertion	Verify word-aligned addressing using PC[31:2]	PASS

Functional Coverage

Coverpoint	Bins Covered
------------	--------------

PC Address Coverage Address 0, Address 1, Address 2, Address 3

Functional Coverage Achieved: 100%

Verification Results

```
# KERNEL:
# KERNEL: TC1 : Address 0
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : ADDR0
# KERNEL:
# KERNEL: TC2 : Address 1
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : ADDR1
# KERNEL:
# KERNEL: TC3 : Address 2
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : ADDR2
# KERNEL:
# KERNEL: TC4 : Address 3
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : ADDR3
# KERNEL:
# KERNEL: TC5 : Alignment Check
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: TC6 : Alignment Check
# KERNEL: PASS : TC6
# KERNEL:
# KERNEL: TC7 : Alignment Check
# KERNEL: PASS : TC7
# KERNEL:
# KERNEL: INSTRUCTION_MEMORY VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

```
# MODULE - work.tb_instruction_memory
#
#
# SUMMARY
# =====
# | Coverage Type | Weight | Hits/Total |
# =====
# | Covergroup Coverage | 1 | 100.000% |
# |-----|-----|
# | Types | 1 / 1 |
# =====
# WEIGHTED AVERAGE: 100.000%
#
#
# COVERGROUP COVERAGE
# =====
# | Covergroup | Hits | Goal / | Status |
# | | | At Least | |
# =====
# | TYPE /tb_instruction_memory/instr_mem_cg | 100.000% | 100.000% | Covered |
# =====
# | INSTANCE INSTR_MEM_COVERAGE | 100.000% | 100.000% | Covered |
# |-----|-----|
# | COVERPOINT INSTR_MEM_COVERAGE::cp_pc | 100.000% | 100.000% | Covered |
# |-----|-----|
# | bin addr0 | 4 | 1 | Covered |
# | bin addr1 | 1 | 1 | Covered |
# | bin addr2 | 1 | 1 | Covered |
# | bin addr3 | 1 | 1 | Covered |
# =====
#
exit
# KERNEL:
# KERNEL: Final Coverage = 100.00%
# VSIM: Simulation has finished.
```

17. MEM/WB Pipeline Register Verification

Module Description

The MEM/WB pipeline register transfers data and control signals from the Memory (MEM) stage to the Write Back (WB) stage. It stores ALU results, memory read data, destination register information, control signals, PC+4 value, and result source selection. The module supports synchronous reset functionality.

Testcases Executed

TC No	Description	Result
TC1	Reset Verification	PASS
TC2	Data Path Transfer Verification	PASS
TC3	Control Path Transfer Verification	PASS
TC4	PC+4 Transfer Verification	PASS
TC5	Result Source Transfer Verification	PASS
TC6	Data Update Verification	PASS
TC7	ALU Path with Control OFF	PASS
TC8	Memory Path Verification	PASS
TC9	Default Path Verification	PASS
TC10	High Register with Control ON	PASS
TC11	Cross Coverage Verification (MEM Path + RegWrite ON)	PASS
TC12	Cross Coverage Verification (PC4 Path + RegWrite OFF)	PASS
TC13	Cross Coverage Verification (Default Path + RegWrite ON)	PASS

Assertions Used

Assertion	Purpose	Result
ALU Transfer Assertion	Verify ALU result transfers correctly to WB stage	PASS
Register Write Assertion	Verify reg_write control signal transfer	PASS
MemToReg Assertion	Verify mem_to_reg control signal transfer	PASS
PC+4 Assertion	Verify PC+4 value transfer	PASS
Result Source Assertion	Verify result_src transfer	PASS

Assertion	Purpose	Result
Destination Register Assertion	Verify destination register transfer	PASS

Functional Coverage

Coverpoint	Bins Covered
Reset Coverage	Asserted, Deasserted
RegWrite Coverage	ON, OFF
MemToReg Coverage	ON, OFF
Result Source Coverage	ALU Path, MEM Path, PC+4 Path, Default Path
Destination Register Coverage	x0, Low Registers, Mid Registers, High Registers
Cross Coverage	Result Source $\times$ RegWrite

Verification Results

```
# KERNEL: PASS : TC1 Reset
# KERNEL: ASSERT PASS : ALU TRANSFER
# KERNEL: PASS : TC2 Data Path
# KERNEL: ASSERT PASS : REG_WRITE
# KERNEL: ASSERT PASS : MEM_TO_REG
# KERNEL: PASS : TC3 Control Path
# KERNEL: ASSERT PASS : PC4
# KERNEL: PASS : TC4 PC+4
# KERNEL: ASSERT PASS : RESULT_SRC
# KERNEL: PASS : TC5 result_src
# KERNEL: ASSERT PASS : RD
# KERNEL: PASS : TC6 Update Test
# KERNEL:
# KERNEL: MEM_WB VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_mem_wb/mem_wb_cg	100.000%	100.000%	Covered
INSTANCE MEM_WB_COVERAGE	100.000%	100.000%	Covered
COVERPOINT MEM_WB_COVERAGE::cp_rst	100.000%	100.000%	Covered
bin asserted	1	1	Covered
bin deasserted	13	1	Covered
COVERPOINT MEM_WB_COVERAGE::cp_reg_write	100.000%	100.000%	Covered
bin off	4	1	Covered
bin on	10	1	Covered
COVERPOINT MEM_WB_COVERAGE::cp_mem_to_reg	100.000%	100.000%	Covered
bin off	3	1	Covered
bin on	11	1	Covered
COVERPOINT MEM_WB_COVERAGE::cp_result_src	100.000%	100.000%	Covered
bin alu_path	2	1	Covered
bin mem_path	2	1	Covered
bin pc4_path	7	1	Covered
bin default_path	3	1	Covered
COVERPOINT MEM_WB_COVERAGE::cp_rd	100.000%	100.000%	Covered
bin x0	1	1	Covered
bin low_reg	6	1	Covered
bin mid_reg	1	1	Covered
bin high_reg	6	1	Covered
CROSS MEM_WB_COVERAGE::cross_result_regwrite	100.000%	100.000%	Covered
bin <alu_path,off>	1	1	Covered
bin <alu_path,on>	1	1	Covered
bin <mem_path,off>	1	1	Covered
bin <mem_path,on>	1	1	Covered
bin <pc4_path,off>	1	1	Covered
bin <pc4_path,on>	6	1	Covered
bin <default_path,off>	1	1	Covered
bin <default_path,on>	2	1	Covered

18. MUX_WB Verification

Module Description

The MUX_WB module selects the final data to be written back to the register file. Based on the result_src control signal, it chooses between the ALU result, memory read data, or PC+4 value. This module is used in the Write Back (WB) stage of the RISC-V pipeline.

Testcases Executed

TC No	Description	Result
TC1	ALU Result Selection Verification	PASS
TC2	Memory Data Selection Verification	PASS
TC3	PC+4 Selection Verification	PASS
TC4	Default Path Verification	PASS
TC5	Dynamic Switching Verification	PASS

Assertions Used

Assertion	Purpose	Result
ALU Path Assertion	Verify ALU result is selected when result_src = 2'b00	PASS
Memory Path Assertion	Verify memory data is selected when result_src = 2'b01	PASS
PC+4 Path Assertion	Verify PC+4 value is selected when result_src = 2'b10	PASS
Default Path Assertion	Verify ALU result is selected for default case (2'b11)	PASS

Functional Coverage

Coverpoint	Bins Covered
Result Source Coverage	ALU Path (00), Memory Path (01), PC+4 Path (10), Default Path (11)

Verification Results

```
# KERNEL:
# KERNEL: TC1 : ALU Result Select
# KERNEL: ASSERT PASS : ALU PATH
# KERNEL: PASS : TC1
# KERNEL:
# KERNEL: TC2 : Memory Data Select
# KERNEL: ASSERT PASS : MEMORY PATH
# KERNEL: PASS : TC2
# KERNEL:
# KERNEL: TC3 : PC+4 Select
# KERNEL: ASSERT PASS : PC4 PATH
# KERNEL: PASS : TC3
# KERNEL:
# KERNEL: TC4 : Default Case
# KERNEL: ASSERT PASS : DEFAULT PATH
# KERNEL: PASS : TC4
# KERNEL:
# KERNEL: TC5 : Dynamic Switching
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: MUX_WB VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

```
# MODULE - work.tb_mux_wb
#
#
# SUMMARY
# =====
# | Coverage Type | Weight | Hits/Total |
# =====
# | Covergroup Coverage | 1 | 100.000% |
# |-----|-----|-----|
# | Types | | 1 / 1 |
# =====
# WEIGHTED AVERAGE: 100.000%
#
#
# COVERGROUP COVERAGE
# =====
# | Covergroup | Hits | Goal / | Status |
# | | | At Least | |
# =====
# | TYPE /tb_mux_wb/mux_wb_cg | 100.000% | 100.000% | Covered |
# =====
# | INSTANCE MUX_WB_COVERAGE | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | COVERPOINT MUX_WB_COVERAGE::cp_result_src | 100.000% | 100.000% | Covered |
# |-----|-----|-----|-----|
# | bin alu_path | 2 | 1 | Covered |
# | bin mem_path | 2 | 1 | Covered |
# | bin pc4_path | 2 | 1 | Covered |
# | bin default_path | 2 | 1 | Covered |
# =====
#
exit
# KERNEL:
# KERNEL: Final Coverage = 100.00%
# VSIM: Simulation has finished.
```

19. MUX Verification

Module Description

The MUX module selects the next Program Counter (PC) value. Based on the PCSrc control signal, it chooses either the sequential address (PC+4) or the branch/jump target address (PCTarget).

Testcases Executed

TC No	Description	Result
TC1	PC+4 Path Verification	PASS
TC2	Target Path Verification	PASS
TC3	Dynamic Input Switching Verification	PASS
TC4	Zero Value Verification	PASS
TC5	Maximum Value Verification	PASS

Assertions Used

Assertion	Purpose	Result
PC+4 Path Assertion	Verify next_pc = pc_plus_4 when PCSrc = 0	PASS
Target Path Assertion	Verify next_pc = PCTarget when PCSrc = 1	PASS

Functional Coverage

Coverpoint Bins Covered

PCSrc Coverage PC+4 Path (0), Target Path (1)

Verification Results

```
# KERNEL: TC1 : PC+4 Path
# KERNEL: ASSERT PASS : PC+4 PATH
# KERNEL: PASS : TC1
# KERNEL:
# KERNEL: TC2 : Target Path
# KERNEL: ASSERT PASS : TARGET PATH
# KERNEL: PASS : TC2
# KERNEL:
# KERNEL: TC3 : Dynamic Inputs
# KERNEL: ASSERT PASS : PC+4 PATH
# KERNEL: PASS : TC3-1
# KERNEL: ASSERT PASS : TARGET PATH
# KERNEL: PASS : TC3-2
# KERNEL:
# KERNEL: TC4 : Zero Values
# KERNEL: ASSERT PASS : PC+4 PATH
# KERNEL: PASS : TC4
# KERNEL:
# KERNEL: TC5 : Max Values
# KERNEL: ASSERT PASS : TARGET PATH
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: MUX VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_mux/mux_cg	100.000%	100.000%	Covered
INSTANCE MUX_COVERAGE	100.000%	100.000%	Covered
COVERPOINT MUX_COVERAGE::cp_pcsrc	100.000%	100.000%	Covered
bin pc_plus_4_path	3	1	Covered
bin target_path	3	1	Covered

20. MUXSRC1 Verification

Module Description

The MUXSRC1 module selects the first ALU operand (src1). Based on the forward_A control signal, it chooses either the register file output (rd1), forwarded ALU result, or forwarded MEM/WB result to resolve data hazards.

Testcases Executed

TC No	Description	Result
TC1	Normal Path Verification (rd1)	PASS
TC2	ALU Forwarding Verification	PASS
TC3	MEM Forwarding Verification	PASS
TC4	Default Path Verification	PASS
TC5	Dynamic Switching Verification	PASS

Assertions Used

Assertion	Purpose	Result
Normal Path Assertion	Verify src1 = rd1 when forward_A = 00	PASS
ALU Forward Assertion	Verify src1 = alu_result when forward_A = 01	PASS
MEM Forward Assertion	Verify src1 = mem_result when forward_A = 10	PASS
Default Path Assertion	Verify default selection returns rd1	PASS
Dynamic Switching Assertions	Verify correct output during runtime control changes	PASS

Functional Coverage

Coverpoint	Bins Covered
forward_A Coverage	Normal Path (00), ALU Forward (01), MEM Forward (10), Default Path (11)

Verification Results

```
# KERNEL:
# KERNEL: TC1 : NORMAL PATH
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : NORMAL PATH
# KERNEL:
# KERNEL: TC2 : ALU FORWARD
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : ALU FORWARD
# KERNEL:
# KERNEL: TC3 : MEM FORWARD
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : MEM FORWARD
# KERNEL:
# KERNEL: TC4 : DEFAULT PATH
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : DEFAULT PATH
# KERNEL:
# KERNEL: TC5 : DYNAMIC SWITCHING
# KERNEL: ASSERT PASS : DYNAMIC NORMAL
# KERNEL: ASSERT PASS : DYNAMIC ALU
# KERNEL: ASSERT PASS : DYNAMIC MEM
# KERNEL: ASSERT PASS : DYNAMIC DEFAULT
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: MUXSRC1 VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

=====			
Covergroup	Hits	Goal /	Status
		At Least	
=====			
TYPE /tb_muxSrc1/muxSrc1_cg	100.000%	100.000%	Covered
=====			
INSTANCE MUXSRC1_COVERAGE	100.000%	100.000%	Covered

COVERPOINT MUXSRC1_COVERAGE::cp_forward_A	100.000%	100.000%	Covered

bin normal_path	2	1	Covered
bin alu_forward	2	1	Covered
bin mem_forward	2	1	Covered
bin default_path	2	1	Covered
=====			

21. MUXSRC2 Verification

Module Description

The MUXSRC2 module selects the second ALU operand (src2). Based on the forward_B control signal, it chooses either the register file output (rd2), forwarded ALU result, or forwarded MEM/WB result to resolve data hazards.

Testcases Executed

TC No	Description	Result
TC1	Normal Path Verification (rd2)	PASS
TC2	ALU Forwarding Verification	PASS
TC3	MEM Forwarding Verification	PASS
TC4	Default Path Verification	PASS
TC5	Dynamic Switching Verification	PASS

Assertions Used

Assertion	Purpose	Result
Normal Path Assertion	Verify src2 = rd2 when forward_B = 00	PASS
ALU Forward Assertion	Verify src2 = alu_result when forward_B = 01	PASS
MEM Forward Assertion	Verify src2 = mem_result when forward_B = 10	PASS
Default Path Assertion	Verify default selection returns rd2	PASS
Dynamic Switching Assertions	Verify correct output during runtime control changes	PASS

Functional Coverage

Coverpoint	Bins Covered
forward_B Coverage	Normal Path (00), ALU Forward (01), MEM Forward (10), Default Path (11)

Verification Results

```
# KERNEL:
# KERNEL: TC1 : NORMAL PATH
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : NORMAL PATH
# KERNEL:
# KERNEL: TC2 : ALU FORWARD
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : ALU FORWARD
# KERNEL:
# KERNEL: TC3 : MEM FORWARD
# KERNEL: PASS : TC3
# KERNEL: ASSERT PASS : MEM FORWARD
# KERNEL:
# KERNEL: TC4 : DEFAULT PATH
# KERNEL: PASS : TC4
# KERNEL: ASSERT PASS : DEFAULT PATH
# KERNEL:
# KERNEL: TC5 : DYNAMIC SWITCHING
# KERNEL: ASSERT PASS : DYNAMIC NORMAL
# KERNEL: ASSERT PASS : DYNAMIC ALU
# KERNEL: ASSERT PASS : DYNAMIC MEM
# KERNEL: ASSERT PASS : DYNAMIC DEFAULT
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: MUXSRC2 VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

=====			
Covergroup	Hits	Goal /	Status
		At Least	
=====			
TYPE /tb_muxSrc2/muxSrc2_cg	100.000%	100.000%	Covered
=====			
INSTANCE MUXSRC2_COVERAGE	100.000%	100.000%	Covered
----- ----- ----- -----			
COVERPOINT MUXSRC2_COVERAGE::cp_forward_B	100.000%	100.000%	Covered
----- ----- ----- -----			
bin normal_path	2	1	Covered
bin alu_forward	2	1	Covered
bin mem_forward	2	1	Covered
bin default_path	2	1	Covered
=====			

22. MUXSRCIMM Verification

Module Description

The MUXSRCIMM module selects the second ALU operand (src2) between the register value (src2_in) and immediate value (imm) based on the alu_src control signal.

Testcases Executed

TC No	Description	Result
TC1	Register Path Verification (src2_in)	PASS
TC2	Immediate Path Verification (imm)	PASS
TC3	Dynamic Switching Verification	PASS
TC4	Zero Values Verification	PASS
TC5	Large Values Verification	PASS

Assertions Used

Assertion	Purpose	Result
Register Path Assertion	Verify src2 = src2_in when alu_src = 0	PASS
Immediate Path Assertion	Verify src2 = imm when alu_src = 1	PASS
Dynamic Register Assertion	Verify correct output during runtime switch to register path	PASS
Dynamic Immediate Assertion	Verify correct output during runtime switch to immediate path	PASS

Functional Coverage

Coverpoint Bins Covered

alu_src Coverage Register Path (0), Immediate Path (1)

Verification Result

```
# KERNEL:
# KERNEL: TC1 : Register Path
# KERNEL: PASS : TC1
# KERNEL: ASSERT PASS : REGISTER PATH
# KERNEL:
# KERNEL: TC2 : Immediate Path
# KERNEL: PASS : TC2
# KERNEL: ASSERT PASS : IMMEDIATE PATH
# KERNEL:
# KERNEL: TC3 : Dynamic Switching
# KERNEL: ASSERT PASS : DYNAMIC REGISTER
# KERNEL: ASSERT PASS : DYNAMIC IMMEDIATE
# KERNEL: PASS : TC3
# KERNEL:
# KERNEL: TC4 : Zero Values
# KERNEL: PASS : TC4
# KERNEL:
# KERNEL: TC5 : Large Values
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: MUXSRCIMM VERIFICATION COMPLETE
# KERNEL:
# KERNEL: Coverage = 100.00%
```

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_muxSrcImm/muxSrcImm_cg	100.000%	100.000%	Covered
INSTANCE MUXSRCIMM_COVERAGE	100.000%	100.000%	Covered
COVERPOINT MUXSRCIMM_COVERAGE::cp_alu_src	100.000%	100.000%	Covered
bin reg_path	3	1	Covered
bin imm_path	3	1	Covered

23. REG_FILE Verification

Module Description

The REG_FILE module implements a 32-register RISC-V register file. It supports synchronous write operations, asynchronous read operations, reset functionality, and permanent protection of register x0.

Testcases Executed

TC No	Description	Result
TC1	Reset Verification	PASS
TC2	Single Register Write Verification	PASS
TC3	Dual Read Verification	PASS
TC4	Write Disable Verification	PASS
TC5	x0 Protection Verification	PASS
TC6	Multiple Register Writes Verification	PASS
TC7	Register Update Verification	PASS
TC8	Coverage Closure Verification	PASS
TC9	Mid Register Coverage Verification	PASS

Assertions Used

Assertion	Purpose	Result
x0 Zero Assertion	Verify register x0 always remains zero	PASS
Register Write Assertion	Verify valid write updates destination register	PASS
x0 Protection Assertion	Verify writes to x0 are ignored	PASS
Reset Clear Assertion	Verify reset clears register contents	PASS

Functional Coverage

Coverpoint	Bins Covered
Reset Coverage	Asserted, Deasserted
Register Write Coverage	Enabled, Disabled
RD Coverage	x0, Low, Mid, High Registers
RS1 Coverage	x0, Low, Mid, High Registers
RS2 Coverage	x0, Low, Mid, High Registers
Write Data Coverage	Low, Medium, High Data Values
Cross Coverage	reg_write × rd, rst × reg_write

Verification Results

```
# KERNEL:
# KERNEL: TC1 : Reset Verification
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : TC1
# KERNEL:
# KERNEL: TC2 : Single Register Write
# KERNEL: PASS : x0 remains zero
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : TC2
# KERNEL:
# KERNEL: TC3 : Dual Read Verification
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : TC3
# KERNEL:
# KERNEL: TC4 : Write Disable Verification
# KERNEL: PASS : Register Write
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : TC4
# KERNEL:
# KERNEL: TC5 : x0 Protection
# KERNEL: PASS : Register Write
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : TC5
# KERNEL:
# KERNEL: TC6 : Multiple Register Writes
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : x1
# KERNEL: PASS : Register Write
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : x2
# KERNEL: PASS : x3
```

```
# KERNEL: TC7 : Register Update
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Update Step 1
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Update Step 2
# KERNEL:
# KERNEL: TC8 : Coverage Closure
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Register Write
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : x0 Protection
# KERNEL:
# KERNEL: TC9 : Mid Register Coverage
# KERNEL: PASS : x0 remains zero
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : rs1 mid_reg
# KERNEL: PASS : rs2 mid_reg
# KERNEL:
# KERNEL: REG_FILE VERIFICATION COMPLETE
# KERNEL: PASS : Register Write
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL: PASS : Register Write
# KERNEL: PASS : x0 Protection
# KERNEL: PASS : Reset Clear
# KERNEL:
# KERNEL: Coverage = 100.00%
```

MODULE - work.tb_reg_file

SUMMARY

=====		
Coverage Type	Weight	Hits/Total
=====		
Covergroup Coverage	1	100.000%
-----	-----	-----
Types		1 / 1
=====		
WEIGHTED AVERAGE: 100.000%		

COVERGROUP COVERAGE

=====			
Covergroup	Hits	Goal /	Status
		At Least	
=====			
TYPE /tb_reg_file/regfile_cg	100.000%	100.000%	Covered
=====			
INSTANCE REGFILE_COVERAGE	100.000%	100.000%	Covered
-----	-----	-----	-----
COVERPOINT REGFILE_COVERAGE::cp_rst	100.000%	100.000%	Covered
-----	-----	-----	-----
bin asserted	2	1	Covered
bin deasserted	19	1	Covered
-----	-----	-----	-----
COVERPOINT REGFILE_COVERAGE::cp_reg_write	100.000%	100.000%	Covered
-----	-----	-----	-----
bin off	8	1	Covered
bin on	13	1	Covered
-----	-----	-----	-----
COVERPOINT REGFILE_COVERAGE::cp_rd	100.000%	100.000%	Covered
-----	-----	-----	-----
bin x0	2	1	Covered
bin low_reg	10	1	Covered
bin mid_reg	5	1	Covered
bin high_reg	4	1	Covered
-----	-----	-----	-----

COVERPOINT REGFILE_COVERAGE::cp_rs1	100.000%	100.000%	Covered
bin x0	4	1	Covered
bin low_reg	10	1	Covered
bin mid_reg	2	1	Covered
bin high_reg	5	1	Covered
COVERPOINT REGFILE_COVERAGE::cp_rs2	100.000%	100.000%	Covered
bin x0	4	1	Covered
bin low_reg	11	1	Covered
bin mid_reg	2	1	Covered
bin high_reg	4	1	Covered
COVERPOINT REGFILE_COVERAGE::cp_data	100.000%	100.000%	Covered
bin data_low	7	1	Covered
bin data_mid	10	1	Covered
bin data_high	4	1	Covered
CROSS REGFILE_COVERAGE::cross_write_rd	100.000%	100.000%	Covered
bin <off,x0>	1	1	Covered
bin <off,low_reg>	3	1	Covered
bin <off,mid_reg>	3	1	Covered
bin <off,high_reg>	1	1	Covered
bin <on,x0>	1	1	Covered
bin <on,low_reg>	7	1	Covered
bin <on,mid_reg>	2	1	Covered
bin <on,high_reg>	3	1	Covered
CROSS REGFILE_COVERAGE::cross_rst_write	100.000%	100.000%	Covered
bin <asserted,off>	1	1	Covered
bin <asserted,on>	1	1	Covered
bin <deasserted,off>	7	1	Covered
bin <deasserted,on>	12	1	Covered

Final Coverage = 100.00%

24. RISC_TOP Verification

Module Description

The RISC_TOP module integrates all pipeline stages of the RISC-V processor including Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). It also incorporates hazard detection, forwarding, branch handling, and pipeline control logic.

Testcases Executed

TC No	Description	Result
TC1	Reset Verification	PASS
TC2	Instruction Fetch Verification	PASS
TC3	Pipeline Flow Verification	PASS
TC4	Register Writeback Verification	PASS
TC5	Memory Access Verification	PASS
TC6	Hazard Detection Verification	PASS
TC7	Forwarding Verification	PASS
TC8	Branch/Jump Verification	PASS

Assertions Used

Assertion	Purpose	Result
Reset PC Assertion	Verify PC resets to zero after reset	PASS
PC Update Assertion	Verify PC updates with next_pc during normal execution	PASS
PC Stall Assertion	Verify PC holds value during pipeline stall	PASS
Branch Flush Assertion	Verify branch/jump causes pipeline flush	PASS
Stall Assertion	Verify hazard detection disables PC update	PASS
EX Forward A Assertion	Verify EX-stage forwarding is enabled correctly	PASS
MEM Forward A Assertion	Verify MEM-stage forwarding is enabled correctly	PASS
Writeback Assertion	Verify valid destination register during WB stage	PASS
Memory Read Assertion	Verify MEM read and write are not active simultaneously	PASS

Functional Coverage

Coverpoint	Bins Covered
PC Source Coverage	Branch Not Taken, Branch Taken
Flush Coverage	Flush Disabled, Flush Enabled
Stall Coverage	Normal Execution, Pipeline Stall
ForwardA Coverage	No Forwarding, EX Forwarding, MEM Forwarding
ForwardB Coverage	No Forwarding, EX Forwarding, MEM Forwarding
Register Write Coverage	Disabled, Enabled
Memory Read Coverage	Disabled, Enabled
Memory Write Coverage	Disabled, Enabled

Verification Result

```
# KERNEL :  
# KERNEL : =====  
# KERNEL : RISC_TOP VERIFICATION COMPLETE  
# KERNEL : =====  
# KERNEL : ASSERT PASS : RESET PC  
# KERNEL : ASSERT PASS : PC UPDATE  
# KERNEL : ASSERT PASS : STALL  
# KERNEL : ASSERT PASS : WRITEBACK  
# KERNEL : ASSERT PASS : RESET PC  
# KERNEL : ASSERT PASS : PC UPDATE  
# KERNEL : ASSERT PASS : STALL  
# KERNEL : ASSERT PASS : WRITEBACK
```

Final Coverage = 100.00%

COVERGROUP COVERAGE

Covergroup	Hits	Goal / At Least	Status
TYPE /tb_risc_top/risc_top_cg	100.000%	100.000%	Covered
INSTANCE RISC_TOP_COVERAGE	100.000%	100.000%	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_pcsrc	100.000%	100.000%	Covered
bin not_taken	109	1	Covered
bin taken	1	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_flush	100.000%	100.000%	Covered
bin no_flush	109	1	Covered
bin flush	1	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_stall	100.000%	100.000%	Covered
bin normal	109	1	Covered
bin stall	1	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_forwardA	100.000%	100.000%	Covered
bin no_forward	105	1	Covered
bin ex_forward	2	1	Covered
bin mem_forward	3	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_forwardB	100.000%	100.000%	Covered
bin no_forward	106	1	Covered
bin ex_forward	3	1	Covered
bin mem_forward	1	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_regwrite	100.000%	100.000%	Covered
bin write_off	11	1	Covered
bin write_on	99	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_memread	100.000%	100.000%	Covered
bin no_read	108	1	Covered
bin read	2	1	Covered
COVERPOINT RISC_TOP_COVERAGE::cp_memwrite	100.000%	100.000%	Covered
bin no_write	109	1	Covered
bin write	1	1	Covered

Future Work

The current verification effort primarily focused on functional correctness through directed test cases, assertions, waveform analysis, and functional coverage. While the design has successfully passed unit-level and integration-level verification, several advanced verification activities remain for future development.

Future work includes:

1. Code Coverage Analysis

- Perform comprehensive code coverage analysis for all modules.
- Measure statement coverage, branch coverage, condition coverage, toggle coverage, and FSM coverage.
- Identify uncovered portions of RTL and create additional test cases to achieve coverage closure.

2. Coverage Closure

- Analyze uncovered bins from functional coverage reports.
- Develop targeted test scenarios to improve verification completeness.
- Correlate functional coverage with code coverage metrics.

3. Constraint Random Verification

- Replace purely directed testing with constrained-random stimulus generation.
- Generate a larger variety of legal and corner-case scenarios automatically.
- Improve bug detection capability and design robustness.

4. Advanced SystemVerilog Verification

- Expand assertion-based verification using additional SystemVerilog Assertions (SVA).
- Implement protocol checking and temporal property verification.
- Introduce scoreboards and reference models for automated result checking.

5. UVM-Based Verification

- Migrate the verification environment toward Universal Verification Methodology (UVM).
- Develop reusable agents, drivers, monitors, sequencers, and scoreboards.
- Enable scalable and industry-standard verification practices.

The completion of these activities will further improve verification confidence and move the processor verification environment closer to industrial verification standards.